



CONNECTION POOL (POOL DE CONEXIONES)

Gestiona y reutiliza conexiones a la base de datos de forma eficiente, segura y escalable.

IDEA CLAVE

En lugar de abrir y cerrar una conexión cada vez que la necesitas, el pool mantiene un conjunto de conexiones listas para usar.

¿QUÉ PROBLEMA RESUELVE?

- ✗ Abrir/cerrar conexiones es costoso.
- ✗ Consume muchos recursos.
- ✗ Mayor tiempo de respuesta.
- ✗ Puede agotar el número máximo de conexiones de la base de datos.
- ✗ Afecta el rendimiento y la escalabilidad.



¿QUÉ ES?

Es un patrón de diseño que mantiene un conjunto de conexiones abiertas a la base de datos en memoria.

Cuando una aplicación necesita una conexión, la toma del pool. Al terminar, la devuelve para que pueda ser reutilizada.

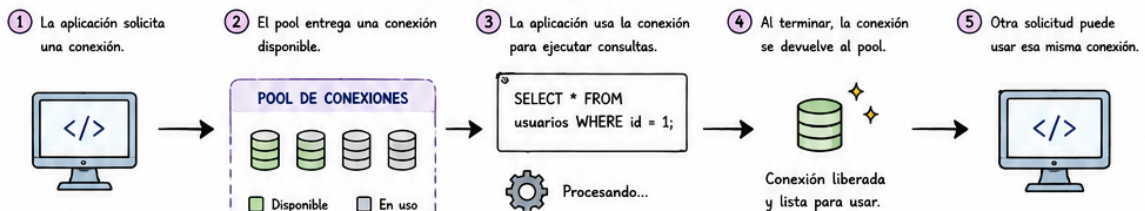


BENEFICIOS



- ✓ Mejor rendimiento y menor latencia.
- ✓ Reduce el consumo de recursos.
- ✓ Mayor escalabilidad.
- ✓ Evita conexiones innecesarias.
- ✓ Control y monitoreo centralizado.
- ✓ Mayor estabilidad del sistema.

¿CÓMO FUNCIONA?



SIN POOL vs CON POOL

SIN POOL (cada petición abre/cierra)

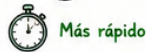
1. Abrir conexión
2. Ejecutar consulta
3. Cerrar conexión



Más lento

CON POOL (reutiliza conexiones)

1. Tomar conexión del pool
2. Ejecutar consulta
3. Devolver al pool

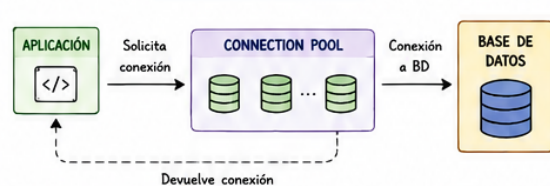


Más rápido

PARÁMETROS COMUNES DE CONFIGURACIÓN

- **initialSize**: Conexiones que se crean al iniciar el pool.
- **minIdle**: Número mínimo de conexiones inactivas que el pool mantiene.
- **maxPoolSize**: Máximo de conexiones que puede crear el pool.
- **maxIdle**: Máximo de conexiones inactivas permitidas.
- **connectionTimeout**: Tiempo máximo para obtener una conexión antes de lanzar error.
- **idleTimeout**: Tiempo que una conexión inactiva puede permanecer en el pool.
- **maxLifetime**: Tiempo máximo de vida de una conexión antes de ser reciclada.

ARQUITECTURA TÍPICA



IMPLEMENTACIONES POPULARES

- HikariCP**: El más rápido y recomendado en Java.
- Apache DBCP**: Implementación robusta de Apache.
- C3PO**: Muy configurable y ampliamente usado.
- Tomcat JDBC Pool**: Ligero e integrado con Tomcat.
- node-postgres pool**: Pool de conexiones para Node.js y PostgreSQL.

BUENAS PRÁCTICAS

- ✓ Usa un pool, nunca abras/cierres conexiones manualmente en cada operación.
- ✓ Ajusta **maxPoolSize** según la capacidad de tu base de datos.
- ✓ Usa timeouts adecuados para evitar bloqueos.
- ✓ Habilita validación de conexiones para evitar conexiones muertas.
- ✓ Monitorea el pool (tamaño, uso, tiempo de espera, errores).
- ✓ Prueba el rendimiento y ajusta según la carga real.



¿DÓNDE SE USA?

- Aplicaciones web
- APIs y microservicios
- Sistemas empresariales
- Procesos batch
- Cualquier app que use BD

EJEMPLO (Java - HikariCP)

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:postgresql://localhost:5432/miapp");
config.setUsername("usuario");
config.setPassword("*****");
config.setMaximumPoolSize(20);
config.setMinimumIdle(5);

HikariDataSource dataSource = new HikariDataSource(config);

// El pool ya está listo para usar!
```



¿QUÉ ES UN ADR?

Un ADR (Architecture Decision Record) es un documento breve que registra una decisión arquitectónica importante, el contexto, las alternativas consideradas, la decisión tomada y sus consecuencias.

¿PARA QUÉ SIRVE?

- ✓ Preservar el conocimiento arquitectónico.
- ✓ Dar contexto a las decisiones tomadas.
- ✓ Facilitar la comunicación del equipo.
- ✓ Onboarding de nuevos integrantes.
- ✓ Auditoría y cumplimiento.
- ✓ Evitar discusiones repetitivas.

PRINCIPIOS

- Escribe lo suficiente, no todo.
- Sé claro y conciso.
- Cada ADR debe ser atómico (una decisión).
- Usa lenguaje entendible para el equipo.
- Actualiza cuando cambie el estado.
- Almacena en un lugar accesible y versionado (ej. repositorio Git).

BUENAS PRÁCTICAS

- Una decisión por ADR. Mantén cada ADR enfocado en una sola decisión.
- Sé claro y breve. Evita documentos largos, escribe lo necesario.
- Involucra al equipo correcto. Incluye a las personas que entienden el contexto y el impacto.
- Almacena en un repositorio. Guárdalos en Git en la carpeta /docs/adr/.
- Usa numeración secuencial. ADR-001, ADR-002, ...
- Revisa y actualiza. Mantén el estado y las referencias actualizadas.

HERRAMIENTAS DONDE GESTIONAR ADRs

Git (Markdown en repositorio), Confluence, Notion, MDocs, Documentación interna.

REGISTRO DE DECISIONES ARQUITECTÓNICAS (ADR)

Documenta de forma clara y estructurada las decisiones importantes que dan forma a la arquitectura del sistema.

ESTRUCTURA DE UN ADR (PLANTILLA)

- TÍTULO**: Nombre corto y descriptivo de la decisión.
- ESTADO**: Propuesto | Aceptado | Reemplazado | Obsoleto | Superseded
- CONTEXTO**: Situación actual y problema que motiva la decisión.
- DECISIÓN**: La decisión tomada y por qué se eligió esta opción.
- CONSECUENCIAS**: Impacto positivo y negativo de la decisión.
- ALTERNATIVAS CONSIDERADAS**: Opciones evaluadas y por qué no fueron elegidas.
- ENLACES / REFERENCIAS**: Enlaces a documentos, issues, diagramas, PR, etc.
- FECHA Y AUTORES**: Cuando se tomó la decisión y quiénes participaron.

EJEMPLO DE ADR

ADR-007 Usar PostgreSQL como base de datos

Estado: **Aceptado**

Fecha: 2024-05-15 Autores: Equipo de Arquitectura

3. Contexto
El sistema requiere una base de datos relacional robusta, escalable y con soporte para datos complejos.

4. Decisión
Se decide usar PostgreSQL como motor de base de datos principal. Se eligió por su confiabilidad, soporte, características avanzadas y comunidad activa.

5. Consecuencias
+ Soporte ACID, JSONB, replicación, extensiones. Licencia open source.
- Mayor curva de aprendizaje vs MySQL.

6. Alternativas consideradas
- MySQL: descartado por limitaciones en tipos de datos avanzados.
- MongoDB: descartado por requerimiento de consistencia ACID.

7. Enlaces / Referencias
- Comparativa BD: [link](#)
- Issue #123
- Diagrama de datos

8. Decisión tomada por: Equipo de Arquitectura
Revisado por: Líder Técnico

¿CUÁNDO CREAR UN ADR?

- ✓ Cuando la decisión tenga impacto significativo.
- ✓ Cuando existan varias opciones válidas.
- ✓ Cuando sea probable que la decisión genere dudas futuras.
- ✓ Cuando afecte atributos de calidad (rendimiento, seguridad, escalabilidad, costos, confiabilidad, etc.).



EJEMPLO DE LISTA DE ADRs

ID	Título	Estado	Fecha	Autores
ADR-001	Elegir arquitectura por capas	Aceptado	2024-03-10	Equipo de Arq.
ADR-002	Usar Docker para contenedores	Aceptado	2024-03-15	Equipo de Arq.
ADR-003	Autenticación con JWT	Aceptado	2024-03-22	Equipo de Arq.
ADR-004	Comunicación asíncrona con Kafka	Propuesto	2024-04-02	Equipo de Arq.
...	...	...	...	...

CHECKLIST RÁPIDO

- ✓ ¿La decisión tiene impacto arquitectónico?
- ✓ ¿Se documentó el contexto y el problema?
- ✓ ¿Se evaluaron alternativas?
- ✓ ¿Se registraron las consecuencias?
- ✓ ¿Se incluyeron referencias y enlaces?
- ✓ ¿Se definió el estado y la fecha?

“Lo que no se documenta, se olvida. Lo que se registra, construye arquitectura.”

¿QUÉ ES?

Define cómo se estructuran los datos, dónde se almacenan, cómo fluyen entre los componentes y sistemas, y cómo se integran con otros servicios o aplicaciones.

OBJETIVOS

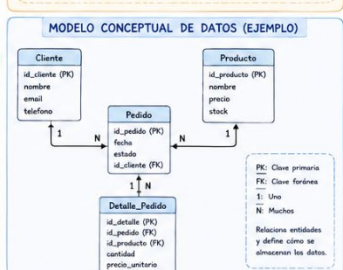
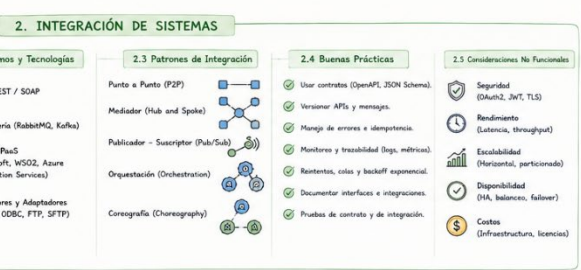
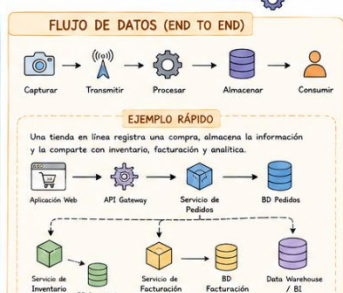
- ✓ Asegurar la calidad e integridad de los datos.
- ✓ Garantizar disponibilidad, seguridad y consistencia.
- ✓ Facilitar el intercambio e integración entre sistemas.
- ✓ Permitir escalabilidad y rendimiento.
- ✓ Soportar la toma de decisiones con datos confiables.

PRINCIPIOS CLAVE

- Un solo origen de verdad (evitar duplicidad).
- Datos normalizados y consistentes.
- Acoplamiento bajo, cohesión alta.
- Seguridad y privacidad por diseño.
- Integración estandarizada.
- Trazabilidad y auditoría.

ARQUITECTURA DE DATOS E INTEGRACIÓN

Organiza, almacena y mueve los datos de forma segura, eficiente y confiable para que los sistemas trabajen juntos y generen valor.



BENEFICIOS

- Datos confiables y consistentes.
- Integración ágil con otros sistemas.
- Reducción de duplicidad y errores.
- Mejor toma de decisiones.
- Escalabilidad y flexibilidad del negocio.

RIESGOS SI NO SE GESTIONA BIEN

- Datos inconsistentes o duplicados.
- Integraciones frágiles y difíciles de mantener.
- Pérdida de información y trazabilidad.
- Problemas de seguridad y cumplimiento.
- Bajo rendimiento y alta indisponibilidad.

HERRAMIENTAS SUGERIDAS

- Bases de datos: MySQL, PostgreSQL, MongoDB, SQL Server
- Almacenamiento: AWS S3, Azure Blob, Google Cloud Storage
- Cache: Redis, Memcached
- Mensajería: RabbitMQ, Apache Kafka
- APIs: REST, gRPC, GraphQL
- Integración: MuleSoft, Apache Camel, WSO2, Talend
- Observabilidad: Prometheus, Grafana, ELK

CHECKLIST RÁPIDO

- ✓ ¿Modelé correctamente mis datos?
- ✓ ¿Mis integraciones son seguras y monitoreables?
- ✓ ¿Tengo manejo de errores y reintentos?
- ✓ ¿Documenté mis APIs y contratos?
- ✓ ¿Probé mis integraciones de extremo a extremo?



¿QUÉ ES?

La vista de despliegue describe dónde se ejecutan los componentes del sistema, qué nodos de hardware los alojan y cómo se comunican a través de la red.

CARACTERÍSTICAS

- ✓ Representa la infraestructura física.
- ✓ Muestra la ubicación de los artefactos (aplicaciones, servicios, bases de datos).
- ✓ Define los canales de comunicación (protocolos, puertos).
- ✓ Ayuda a planificar el rendimiento, escalabilidad y seguridad.
- ✓ Facilita la implementación y el mantenimiento.

ELEMENTOS PRINCIPALES

- Nodo:** recurso físico donde se ejecutan los componentes (servidor, dispositivo, contenedor, etc.).
- Artefacto:** pieza física del software (archivo .war, .jar, .exe, .dll, contenedor, script, etc.).
- Componente:** unidad lógica que proporciona funcionalidad.
- Conexión:** canal de comunicación entre nodos.
- Dependencia:** relación de uso entre artefactos o componentes.

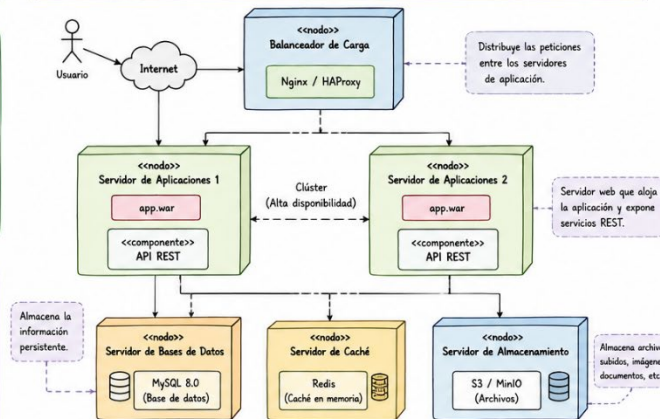
RECUERDA

La vista de despliegue complementa la vista de componentes mostrando DÓNDE se ejecuta cada elemento y CÓMO se comunica.

VISTA DE DESPLIEGUE

Muestra la distribución física del software en los nodos de hardware y las conexiones entre ellos en tiempo de ejecución.

EJEMPLO - SISTEMA DE GESTIÓN DE PEDIDOS (COMERCIO ELECTRÓNICO)



TIPOS DE CONEXIONES COMUNES

- HTTP/HTTPS (80/443): Comunicación web segura entre cliente y servidor.
- TCP/IP (3306): Conexión a base de datos (MySQL, PostgreSQL, etc.).
- TCP/IP (6379): Conexión a caché (Redis, Memcached).
- S3 API / HTTPS: Acceso a almacenamiento de objetos.

BUENAS PRÁCTICAS

- Separar entornos (Desarrollo, Pruebas, Producción).
- Usar balanceadores y clústeres para alta disponibilidad.
- Asegurar las comunicaciones (HTTPS, VPN).
- Monitorizar recursos (CPU, memoria, red, almacenamiento).
- Documentar puertos, protocolos y dependencias.

HERRAMIENTAS PARA MODELAR

- diagrams.net (draw.io)
- Visual Paradigm
- StarUML
- Enterprise Architect
- Lucidchart
- PlantUML

NOTA

Esta vista representa el estado en tiempo de ejecución (runtime) del sistema.

LEYENDA

- Nodo:** Recurso físico de hardware.
- Artefacto:** Archivo físico desplegado en un nodo.
- Componente:** Unidad modular de software.
- Conexión:** Enlace de comunicación entre nodos.
- Dependencia (uso):** Un componente o artefacto utiliza otro.
- Base de datos / Recurso:** Recurso persistente externo.

¿PARA QUÉ SIRVE?

- ✓ Planificar la infraestructura del sistema.
- ✓ Identificar requisitos de hardware y red.
- ✓ Garantizar disponibilidad y tolerancia a fallos.
- ✓ Apoyar decisiones de escalabilidad.
- ✓ Documentar el entorno de producción.



¿QUÉ ES?

La vista de componentes muestra la estructura del sistema a nivel de componentes de software, sus interfaces públicas y cómo se relacionan entre sí para cumplir con los requerimientos.

CARACTERÍSTICAS

- ✓ Agrupa la lógica del sistema en componentes reutilizables.
- ✓ Expone y define interfaces.
- ✓ Muestra dependencias (uso) entre componentes.
- ✓ Promueve el bajo acoplamiento y la alta cohesión.
- ✓ Independiente de la tecnología de implementación.

ELEMENTOS PRINCIPALES

- Componentes:** módulos de software que agrupan funcionalidades.
- Interfaces:** contratos que exponen o requieren servicios.
- Conectores:** vínculos entre componentes e interfaces.
- Dependencias:** relación de uso entre componentes.
- Recursos:** bases de datos, servicios externos, archivos, dispositivos, etc.

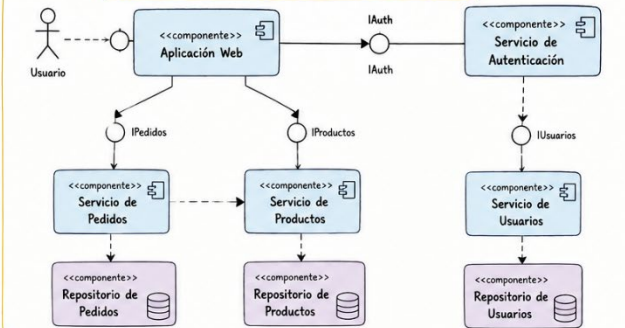
EJEMPLOS DE USO

- Aplicaciones empresariales modulares
- Sistemas con arquitectura por capas
- Microservicios (cada servicio es un componente)
- Integración con servicios externos (APIs)

VISTA DE COMPONENTES

Describe la organización y las dependencias entre los componentes de un sistema de software.

EJEMPLO - SISTEMA DE GESTIÓN DE PEDIDOS



BUENAS PRÁCTICAS

- Alta cohesión:** Cada componente debe tener una única responsabilidad bien definida.
- Bajo acoplamiento:** Minimiza las dependencias entre componentes.
- Interfaces claras:** Define interfaces pequeñas, estables y bien documentadas.
- Reutilización:** Diseña componentes reutilizables en diferentes partes del sistema.
- Independencia:** Permite reemplazar o actualizar componentes sin afectar todo el sistema.

RELACIÓN CON OTRAS VISTAS (4+1)

- Vista Lógica:** Define clases y relaciones del sistema.
- Vista de Proceso:** Muestra procesos e hilos en ejecución.
- Vista de Despliegue:** Muestra nodos donde se implementan los componentes.
- Vista de Casos de Uso:** Representa las funcionalidades del sistema.

HERRAMIENTAS PARA MODELAR

- diagrams.net (draw.io)
- Visual Paradigm
- StarUML
- Enterprise Architect
- Lucidchart
- Asth

LEYENDA

- Componente:** Unidad de implementación con funcionalidad encapsulada.
- Interfaz:** (Interfaz provista/requerida) Punto de interacción entre componentes.
- Conector de ensamblaje:** Conecta un componente con una interfaz.
- Dependencia (uso):** Un componente utiliza los servicios de otro.
- Recurso / Base de datos:** Componentes que representan recursos persistentes externos.

¿PARA QUÉ SIRVE?

- Visualizar la estructura del sistema a nivel modular.
- Identificar responsabilidades de cada componente.
- Analizar dependencias y puntos de integración.
- Facilitar el diseño, la construcción y el mantenimiento.
- Soportar decisiones de reutilización y escalabilidad.



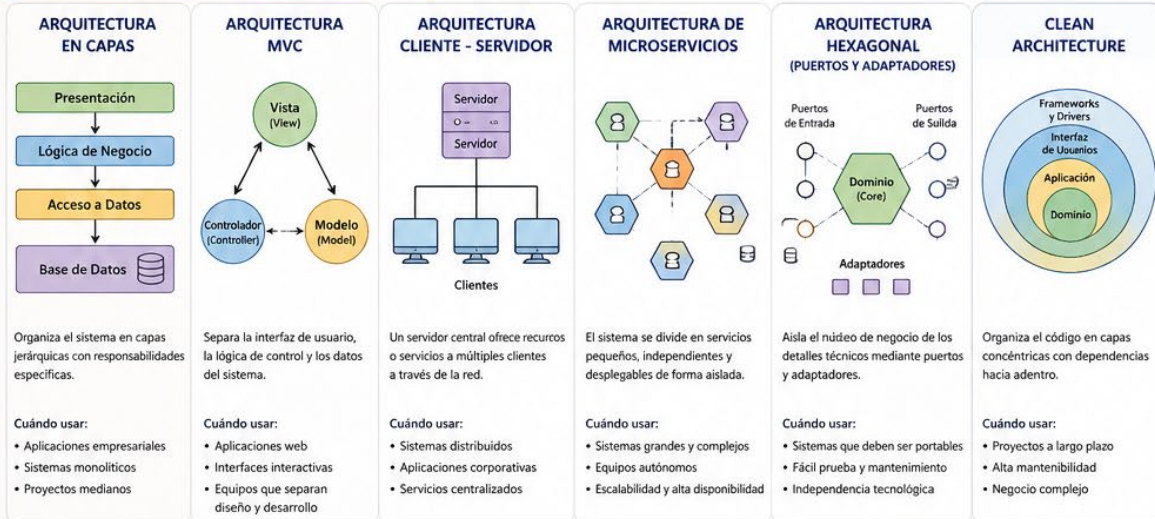
ESTILOS ARQUITECTÓNICOS Y PATRONES DE DISEÑO



Guía rápida para elegir la mejor solución según las necesidades del sistema.

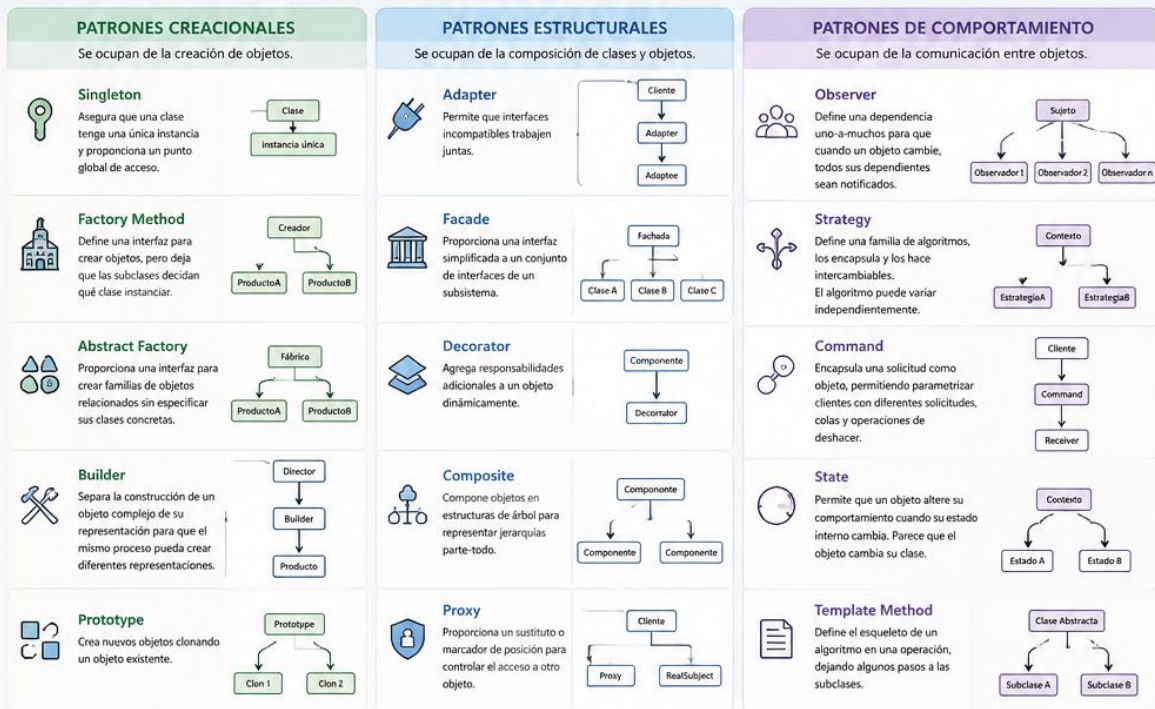
ESTILOS ARQUITECTÓNICOS

Definen la estructura general de un sistema, sus componentes, relaciones y cómo se comunican.



PATRONES DE DISEÑO

Soluciones reutilizables a problemas comunes en el diseño de software. No son código, son ideas y plantillas probadas.



¿CÓMO ELEGIR?

- ✓ Analiza los requerimientos funcionales y no funcionales.
- ✓ Considera el tamaño del sistema y el equipo de desarrollo.
- ✓ Evalúa la escalabilidad, mantenibilidad y despliegue.
- ✓ No existe una única respuesta correcta.
- ✓ La experiencia y el contexto son clave.



RECUERDA

Los estilos arquitectónicos definen la estructura global del sistema.
Los patrones de diseño resuelven problemas específicos dentro del diseño.
Úsalos correctamente para construir software de calidad.

LEYENDA

- Servicio / Componente
- Base de Datos
- Puerto / Interfaz
- Adaptador / Implementación
- Comunicación / Dependencia



CLASES INMUTABLES Y CÓMO VOLVERLAS MUTABLES



¿QUÉ ES?

Una clase inmutable es aquella cuyos objetos NO pueden cambiar su estado después de ser creados.



Una vez creado, su estado permanece siempre igual.

CARACTERÍSTICAS DE UNA CLASE INMUTABLE

- ✓ La clase puede declararse final.
- ✓ Los atributos deben ser private.
- ✓ Los atributos deben ser final.
- ✓ Los valores se asignan en el constructor.
- ✓ No deben existir métodos set.
- ✓ Cuidado con objetos mutables como atributos.
- ✓ Usar copias defensivas cuando sea necesario.

EJEMPLO INMUTABLE

```
public final class Producto {  
    private final String codigo;  
    private final String nombre;  
    private final double precio;  
  
    public Producto(String codigo,  
                    String nombre, double precio){  
        this.codigo = codigo;  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
  
    public String getCodigo(){ return codigo; }  
    public String getNombre(){ return nombre; }  
    public double getPrecio(){ return precio; }  
}
```



No tiene setters → El estado NO puede cambiar.

¿POR QUÉ NO SE PUEDE MODIFICAR?

```
Producto p = new Producto(  
    "P001", "Teclado", 85000  
);  
  
✗ p.setPrecio(70000);  
// Error: el método setPrecio()  
// no existe
```

El compilador no permite cambiar el estado porque NO existen métodos que lo permitan.

BENEFICIOS DE LA INMUTABILIDAD



Seguridad del estado (no cambia inesperadamente).



Simplifica el diseño y el razonamiento del código.



Hilos (threads) más seguros (thread-safe).



Ideal para cachés, valores, DTOs, configuraciones.

¿CÓMO VOLVER UNA CLASE INMUTABLE EN MUTABLE?

ANTES: INMUTABLE

```
private final String nombre;  
private final double precio;  
  
public String getNombre(){...}  
public double getPrecio(){...}
```



- ✗ Sin setters
- ✗ Atributos final
- ✗ Estado NO cambia



QUITAR final
AGREGAR setters
PERMITE cambiar el estado

DESPUÉS: MUTABLE

```
private String nombre;  
private double precio;  
  
public String getNombre(){...}  
public double getPrecio(){...}  
  
public void setNombre(String nombre){  
    this.nombre = nombre;  
}  
public void setPrecio(double precio){  
    this.precio = precio;  
}
```



AHORA SÍ:

- ✓ Tiene setters
- ✓ Atributos NO son final
- ✓ El estado puede cambiar
- ✓ Mayor flexibilidad, pero menos seguridad del estado.

EL PROBLEMA DE OBJETOS MUTABLES



Aunque Empleado sea inmutable, si Direccion cambia, el estado interno también cambia.

SOLUCIÓN: COPIA DEFENSIVA

- En el constructor:
`this.direccion = new Direccion(direccion.getCiudad());`
- En el getter:
`return new Direccion(direccion.getCiudad());`

COMPARATIVA RÁPIDA

ASPECTO	INMUTABLE	MUTABLE
Clase	final (opcional)	normal (opcional)
Atributos	private final	private
Setters	No	Sí
Estado cambia	No	Sí
Seguridad	Alta	Menor
Flexibilidad	Menor	Mayor
Rendimiento	Puede ser mejor en algunos casos	Depende del uso
Uso típico	Valores, DTOs, config, cache, fechas, dinero	Entidades, modelos de negocio, datos editables

EJEMPLO: CUENTA BANCARIA

INMUTABLE

```
public final class Cuenta {  
    private final String numero;  
    private final String titular;  
    private final double saldo;  
  
    // sin setters  
}
```



MUTABLE

```
public class Cuenta {  
    private String numero;  
    private String titular;  
    private double saldo;  
  
    // con setters  
    // y métodos de negocio  
    public void depositar(double v){...}  
    public void retirar(double v){...}  
}
```



REGLAS EN MÉTODOS DE NEGOCIO

- No permitir valores negativos.
- Validar antes de modificar el estado.
- Ejemplo: depositar(-500) → Error



★ RECUERDA

Inmutabilidad NO es solo usar final. Se trata de cuidar TODO el estado interno para que NADIE pueda cambiarlo.



Elige inmutables cuando necesites seguridad y consistencia. Usa mutables cuando necesites flexibilidad y cambios controlados.





BUENAS PRÁCTICAS DE DESARROLLO DDD (DOMAIN-DRIVEN DESIGN)

Construye software alineado al negocio, flexible, mantenible y con complejidad controlada



1. PRINCIPIOS CLAVE DE DDD



Enfócate en el Dominio

Comprende profundamente el dominio del negocio y refleja ese conocimiento en el modelo.



Colabora estrechamente

El equipo de desarrollo y los expertos del dominio deben usar un lenguaje común (Ubiquitous Language).



Modelo rico

El modelo de dominio incluye comportamiento (entidades, valores, servicios) y no solo datos.



Aísla el Dominio

El dominio debe estar independiente de tecnologías y detalles de infraestructura.

2. LENGUAJE UBICUO (UBIQUITOUS LANGUAGE)



Usa un lenguaje común y compartido entre el equipo técnico y el negocio.

- Nombres claros y consistentes.
- Evita jergas técnicas innecesarias.
- Refleja conceptos reales del negocio.

Ejemplo

En un dominio de pedidos:

- ✓ Cliente
- ✓ Pedido
- ✓ Línea de Pedido
- ✓ Estado de Pedido
- ✓ Política de Descuento

3. PATRONES Y PRÁCTICAS ESENCIALES

ENTIDADES



Objetos con identidad única que evolucionan en el tiempo.

Buenas prácticas

- Usa un identificador único.
- Encapsula su comportamiento.
- No expongas setters públicos.

OBJETOS DE VALOR



Inmutables, sin identidad. Se definen por sus atributos.

Buenas prácticas

- Hazlos inmutables.
- Implementa igualdad por valor (todos sus atributos).
- Úsalos para conceptos del dominio (ej. Dinero, Dirección).

AGREGADOS



Conjunto de objetos del dominio tratados como una unidad.

Buenas prácticas

- Define un Agregado Root.
- Protege la integridad del agregado.
- Accede a los elementos internos solo a través del root.

REPOSITORIOS



Colección que ofrece acceso a agregados para persistencia.

Buenas prácticas

- Trabaja con agregados, no con entidades.
- Usa interfaces.
- Evita filtrar lógica del dominio hacia afuera.

SERVICIOS DE DOMINIO



Lógica del dominio que no pertenece a una entidad ni a un objeto de valor.

Buenas prácticas

- Úsalos para operaciones complejas del dominio.
- Sin estado.
- Mantén el dominio expresivo y cohesivo.

EVENTOS DE DOMINIO



Hechos significativos que ocurrieron en el dominio.

Buenas prácticas

- Usa eventos para desacoplar procesos.
- Nómbralos en pasado (ej. PedidoCreado).
- Publica eventos desde el dominio.

4. DISEÑO DE CONTEXTO ACOTADO (BOUNDED CONTEXT)



- ✓ Divide el sistema en contextos según subdominios.
- ✓ Cada contexto tiene su propio modelo y lenguaje.
- ✓ Comunícate entre contextos con contratos bien definidos (APIs, eventos, ACL).
- ✓ Evita compartir modelos entre contextos.

5. CAPAS RECOMENDADAS (ARQUITECTURA)



INTERFAZ DE USUARIO
(Controllers, APIs)



APLICACIÓN
(Casos de uso, orquestación)



DOMINIO
(Modelo del dominio)



INFRAESTRUCTURA
(Persistencia, mensajería, etc.)

- ✓ El dominio ocupa el centro de la arquitectura.
- ✓ Las dependencias apuntan hacia el dominio (regla de dependencia).
- ✓ La infraestructura implementa detalles externos.
- ✓ Mantén el dominio libre de frameworks.

6. OTRAS BUENAS PRÁCTICAS

- ✓ Modela comportamientos, no solo datos.
- ✓ Mantén el modelo simple, pero expresivo.
- ✓ Evita anemias de dominio (entidades sin comportamiento).
- ✓ Usa test automatizados para proteger la lógica del dominio.
- ✓ Refactoriza el modelo continuamente junto con el dominio.
- ✓ Documenta el lenguaje ubicuo.
- ✓ Revisa y alinea el modelo con el negocio de forma continua.

7. ERRORES COMUNES A EVITAR

- ✗ Modelar según la base de datos.
- ✗ Usar DTOs/Anemias para todo.
- ✗ Ignorar el lenguaje ubicuo.
- ✗ Crear un modelo único gigante para todo el sistema.
- ✗ Filtrar la lógica del dominio en servicios de aplicación.
- ✗ Exponer entidades internas de los agregados.
- ✗ Acoplar el dominio a frameworks o tecnologías.

8. BENEFICIOS DE APLICAR DDD



Alineación real
entre negocio
y tecnología



Código más
mantenible y
evolucionable



Menor complejidad
accidental



Equipos más
productivos y
autónomos



Mayor capacidad
de adaptación
al cambio



DDD no es solo diseño, es pensar y construir software que refleja el negocio y genera valor.

CÓMO OPTIMIZAR EL RENDIMIENTO DE LAS APLICACIONES

Mejores prácticas para aplicaciones más rápidas, escalables y eficientes

¿POR QUÉ ES IMPORTANTE?

Un buen rendimiento mejora la **experiencia del usuario**, aumenta la **conversión**, reduce **costos operativos** y permite **escalar el negocio**.

BENEFICIOS CLAVE



Mejor experiencia de usuario



Mayor conversión y retención



Menores costos de infraestructura



Mayor estabilidad y confiabilidad



Escalabilidad sostenible

ESTRATEGIAS CLAVE

1. OPTIMIZA EL CÓDIGO



- Escribe código limpio y eficiente.
- Evita bucles innecesarios y operaciones costosas.
- Usa estructuras de datos adecuadas.
- Reutiliza código y aplica patrones de diseño.

2. OPTIMIZA LAS CONSULTAS A LA BASE DE DATOS



- Usa índices correctamente.
- Evita SELECT *.
- Optimiza JOINs y subconsultas.
- Implementa paginación.
- Revisa y analiza planes de ejecución.

3. USA CACHÉ DE MANERA INTELIGENTE



- Almacena datos frecuentes en caché (Memcached, Redis).
- Define políticas de expiración adecuadas.
- Usa caché en capas (aplicación, datos, CDN).

4. REDUCE PETICIONES Y TRANSFERENCIAS



- Minimiza llamadas a APIs y servicios externos.
- Usa compresión (Gzip, Brotli).
- Reduce el tamaño de las respuestas.
- Implementa carga diferida (lazy loading).

5. DISEÑO Y ARQUITECTURA EFICIENTES



- Aplica arquitectura escalable (microservicios, serverless).
- Desacopla componentes.
- Usa colas y procesamiento asíncrono para tareas pesadas.

6. OPTIMIZA RECURSOS DEL SERVIDOR



- Ajusta la configuración del servidor (CPU, memoria, hilos).
- Usa balanceo de carga.
- Monitorea el uso de recursos y elimina cuellos de botella.

7. MONITOREA Y MIDE CONSTANTEMENTE



- Usa herramientas de monitoreo (APM) como New Relic, Datadog, AppDynamics.
- Define métricas clave: tiempo de respuesta, throughput, uso de recursos, errores.
- Detecta y corrige problemas proactivamente.

8. PRUEBAS DE RENDIMIENTO PERIÓDICAS



- Realiza pruebas de carga, estrés y resistencia.
- Simula escenarios reales de uso.
- Valida cambios antes de pasar a producción.

HERRAMIENTAS RECOMENDADAS

- APM / Monitoreo:** New Relic, Datadog, Dynatrace
- Caché:** Redis, Memcached
- Base de datos:** MySQL, PostgreSQL, MongoDB (con índices y tuning)
- Pruebas de carga:** JMeter, k6, Gatling
- Análisis de código:** SonarQube, ESLint, Checkstyle
- CDN:** Cloudflare, AWS CloudFront

MÉTRICAS CLAVE A MONITOREAR



PROCESO CONTINUO DE OPTIMIZACIÓN



CHECKLIST RÁPIDO

- ✓ ¿El código es eficiente y legible?
- ✓ ¿Las consultas a BD están optimizadas?
- ✓ ¿Se está usando caché adecuadamente?
- ✓ ¿Las peticiones y respuestas están optimizadas?
- ✓ ¿La arquitectura es escalable y resiliente?
- ✓ ¿Se monitorean métricas clave?
- ✓ ¿Se realizan pruebas de rendimiento periódicas?



Optimizar el rendimiento no es una tarea única, es un ciclo constante que genera mejores experiencias, menor costo y mayor competitividad.



CONSULTAS DE GRAN ESCALA CON SQL

¿QUÉ SON?



Son consultas que procesan grandes volúmenes de datos (millones o más de filas). El objetivo es obtener resultados rápidos y eficientes usando buenas prácticas de diseño, escritura y optimización en SQL.

DESAFÍOS PRINCIPALES



Tiempos de respuesta altos



Uso excesivo de CPU, memoria y disco



Bloqueos y contención de recursos



Altos costos de infraestructura

1. BUENAS PRÁCTICAS PARA CONSULTAS DE GRAN ESCALA



1. SELECCIONA SOLO LO NECESARIO

Evita `SELECT *`, selecciona solo las columnas que realmente usas.

-- ❌ Malo
`SELECT * FROM ventas;`
-- ✅ Bueno
`SELECT id, fecha, total FROM ventas;`



2. FILTRA LO ANTES POSIBLE

Usa `WHERE` para reducir el conjunto de datos cuanto antes.

```
SELECT * FROM pedidos
WHERE fecha >= '2024-01-01'
AND estado = 'COMPLETADO';
```



3. USA ÍNDICES

Los índices aceleran las búsquedas y filtros.

```
CREATE INDEX idx_fecha
ON pedidos(fecha);
```

★ Índices en columnas usadas en `WHERE`, `JOIN`, `ORDER BY` y `GROUP BY`.



4. OPTIMIZA JOINS

Asegúrate de unir por columnas indexadas y usa el tipo de `JOIN` adecuado.

```
SELECT c.nombre, SUM(v.total)
FROM clientes c
INNER JOIN ventas v
ON c.id = v.cliente_id
GROUP BY c.nombre;
```



5. AGREGA Y ORDENA CON INTELIGENCIA

Usa `GROUP BY` y `ORDER BY` solo cuando sea necesario. Ordenar grandes volúmenes es costoso.

```
SELECT producto, SUM(total)
FROM ventas
GROUP BY producto;
```



6. USA PÁGINACIÓN

Para resultados grandes, limita y obtén por partes.

```
SELECT * FROM productos
ORDER BY id
LIMIT 10000 OFFSET 10000;
```

★ Evita traer todo el resultado a la vez.



7. PARTICIONA TABLAS

Divide tablas muy grandes por rangos (fecha, región, etc.). Mejora consultas y mantenimiento.

```
CREATE TABLE ventas (
  id BIGINT,
  fecha DATE,
  total DECIMAL(10,2)
) PARTITION BY RANGE (YEAR(fecha));
```



8. MANTIÉN TUS DATOS

Actualiza estadísticas, reorganiza índices y elimina datos innecesarios.

```
ANALYZE TABLE ventas;
OPTIMIZE TABLE ventas;
```

2. HERRAMIENTAS DE APOYO



EXPLAIN / EXPLAIN ANALYZE
Analiza el plan de ejecución.



MONITOREO DE CONSULTAS
Identifica consultas lentas y cuellos de botella.



ÍNDICES COMPUESTOS
Útiles cuando filtras por varias columnas.



VISTAS MATERIALIZADAS
Almacenan resultados de consultas costosas para reutilizarlos.

3. EJEMPLO: ANTES vs DESPUÉS

❌ ANTES (INEFICIENTE)

```
SELECT * FROM ventas v
JOIN clientes c ON v.cliente_id = c.id
WHERE YEAR(v.fecha) = 2024
ORDER BY v.fecha DESC;
```

- ❌ Selecciona todas las columnas
- ❌ No usa rango de fechas
- ❌ Función sobre columna (`YEAR`)
- ❌ Sin índice adecuado
- ❌ Ordena todo el resultado

✅ DESPUÉS (OPTIMIZADO)

```
SELECT v.id, v.fecha, v.total, c.nombre
FROM ventas v
JOIN clientes c ON v.cliente_id = c.id
WHERE v.fecha >= '2024-01-01'
AND v.fecha < '2025-01-01'
ORDER BY v.fecha DESC
LIMIT 1000;
```

- ✅ Solo columnas necesarias
- ✅ Rango de fechas (usa índice)
- ✅ Permite uso de índice en fecha
- ✅ Ordena menos filas (paginación)

4. ESCALA AÚN MÁS



REPLICACIÓN
Distribuye lecturas en réplicas.



SHARDING
Divide datos entre múltiples servidores.



ALMACENAMIENTO EN COLUMNAS
Ideal para análisis (`CLICKHOUSE`, `BIGQUERY`, `SNOWFLAKE`, etc.).



CACHÉ DE CONSULTAS
Reduce la carga consultando resultados ya calculados.

5. CHECKLIST RÁPIDO

- | | |
|--|---|
| ☐ ¿Selecciono solo lo necesario? | ☐ ¿Evito ordenar y agrupar sin necesidad? |
| ☐ ¿Filtro los datos lo antes posible? | ☐ ¿Uso paginación para grandes resultados? |
| ☐ ¿Estoy usando índices adecuados? | ☐ ¿Mis tablas están particionadas si es necesario? |
| ☐ ¿Mis JOINS están optimizados? | ☐ ¿Mantengo estadísticas e índices actualizados? |

REGLA DE ORO

★ Consulta menos datos, lo más pronto posible, de la mejor forma posible.



Escribir buenas consultas no es solo SQL... es estrategia, análisis y conocimiento de tus datos.



MICROSERVICIOS, PATRONES Y BASES DE DATOS



Diseña por dominios,
construye
con patrones,
almacena con
inteligencia.

★ Aplicaciones escalables, mantenibles y alineadas al negocio

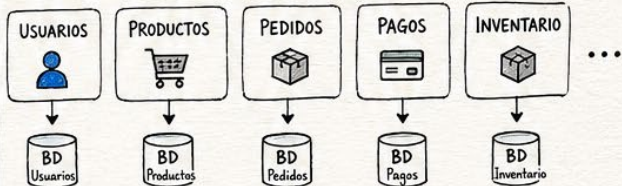
1. ¿QUÉ SON LOS MICROSERVICIOS?

Es un estilo de arquitectura donde una aplicación se construye como un conjunto de servicios pequeños, independientes y especializados en una sola funcionalidad (dominio).



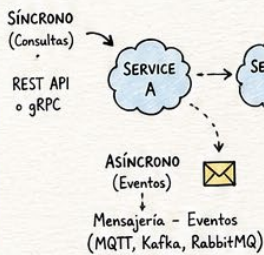
2. MICROSERVICIOS POR DOMINIOS

Divide tu sistema en subdominios del negocio.



★ Cada microservicio tiene su propia base de datos.
No se comparte la BD entre servicios.

3. CÓMO SE COMUNICAN?



! CUÁNDO USAR CADA UNO?

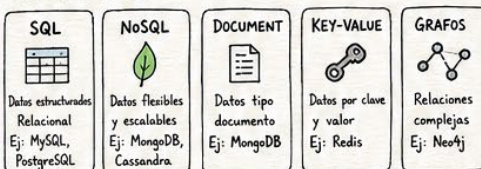
- REST: consultas, operaciones inmediatas
- Eventos: procesos desacoplados, notificaciones, integraciones.

4. PATRONES DE DISEÑO CLAVE ★

- API GATEWAY**: Punto único de entrada. Enruta y gestiona peticiones.
- BULKHEAD**: Aísla recursos para que fallos no afecten todo el sistema.
- CIRCUIT BREAKER**: Evita caídas en cascada cuando un servicio falla.
- SAGA**: Maneja transacciones distribuidas entre varios servicios.
- RETRY**: Reintenta operaciones que pueden fallar temporalmente.
- PLEX/OBSERVER**: Publica eventos y otros servicios se suscriben y reaccionan.

5. BASES DE DATOS EN MICROSERVICIOS

Cada servicio usa la tecnología que mejor se adapta a sus necesidades.



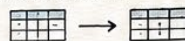
REGLAS DE ORO:

- ✓ Una base de datos por microservicio.
- ✓ Sin joins entre bases de datos de servicios distintos.
- ✓ Cada servicio es dueño de sus datos.
- ✓ Comunicación solo por ID o eventos.

6. PATRONES DE DISEÑO DE BASES DE DATOS

Modelado de Datos

- Entidad - Relación (ER)
- Normalización
- Desnormalización Controlada
- Tabla de Asociación



Estructura

- Single Table
- Class Table Inheritance
- Table per Concrete Class
- Tipos de Datos Especializados



Rendimiento y Escalabilidad

- Indexación
- Particionamiento (Sharding)
- Replicación
- Caching de Datos



Integración y Arquitectura

- Repository Pattern
- CQRS
- Event Sourcing
- Data Warehouse



7. ARQUITECTURAS RELACIONADAS



8. FLUJO GENERAL



BUENAS PRÁCTICAS

- Diseña por dominios, no por tablas.
- Mantén servicios pequeños y con propósito único.
- Usa eventos para desacoplar.
- Automatiza pruebas y despliegues (CI/CD).
- Monitorea, mide y mejora continuamente.



♥ No se trata de la tecnología,
Se trata de resolver problemas
del negocio de la mejor manera.

#SENA #Innovación #AprenderHaciendo



PATRONES DE DISEÑO DE BASES DE DATOS

Soluciones probadas para problemas comunes de modelado de datos



¿QUÉ SON?

Son soluciones reutilizables a problemas recurrentes en el diseño de bases de datos. Nos ayudan a crear modelos más flexibles, mantenibles, escalables y alineados al negocio.



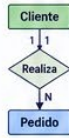
BENEFICIOS

- ✓ Reutilización de soluciones
- ✓ Menos errores de diseño
- ✓ Facilitan el mantenimiento
- ✓ Mejor escalabilidad
- ✓ Alineación con el negocio

PATRONES CLÁSICOS DE MODELADO DE DATOS

1. ENTIDAD-RELACIÓN (ER MODEL)

Representa el mundo real con entidades, atributos y relaciones.



Útil en el análisis inicial y comunicación con el negocio.

2. NORMALIZACIÓN

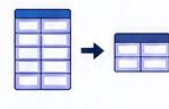
Organiza los datos en tablas evitando redundancias y anomalías.



Formas normales (1FN, 2FN, 3FN, BCNF, 4FN, 5FN).

3. DESNORMALIZACIÓN CONTROLADA

Reduce uniones (JOINS) para mejorar el rendimiento en lecturas.



Útil en reportes, data warehouses y consultas complejas.

4. TABLA DE ASOCIACIÓN

Resuelve relaciones muchos a muchos.



Almacena las claves foráneas de ambas entidades.

PATRONES DE ESTRUCTURA DE DATOS

5. SINGLE TABLE

Toda la información en una sola tabla.



- ✓ Simple y rápido.
- ✗ Pobre escalabilidad y redundancia.

6. CLASS TABLE INHERITANCE (CTI)

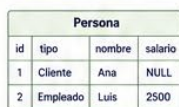
Cada subtipo tiene su propia tabla con la clave del padre.



- ✓ Sin valores nulos.
- ✗ Más uniones (JOINS).

7. SINGLE TABLE INHERITANCE (STI)

Todos los subtipos en una sola tabla con un campo tipo.



- ✓ Menos tablas y uniones.
- ✗ Muchos valores nulos.

8. CONCRETE TABLE INHERITANCE (CTI)

Cada subtipo tiene su tabla con todos los campos.



- ✓ Rendimiento en lecturas.
- ✗ Más almacenamiento.

9. TIPOS DE DATOS ESPECIALIZADOS

Usa tipos propios del motor para mayor integridad.



- ✓ Mayor integridad y validación.
- ✗ Menor portabilidad.

10. TABLA DE AUDITORÍA

Registra cambios (quién, cuándo, qué, valor anterior y nuevo).



- ✓ Trazabilidad completa.
- ✗ Aumenta el volumen de datos.

PATRONES DE RENDIMIENTO Y ESCALABILIDAD

11. PARTICIONAMIENTO

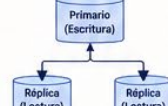
Divide grandes tablas en partes más pequeñas.



- ✓ Mejora rendimiento.
- ✗ Más complejidad en consultas y mantenimiento.

12. REPLICACIÓN

Copia de datos en múltiples servidores.



- ✓ Alta disponibilidad.
- ✗ Consistencia eventual.

13. SHARDING

Distribuye datos horizontalmente en múltiples bases.



- ✓ Escala horizontalmente.
- ✗ Consultas globales complejas.

14. DENORMALIZED READ MODEL

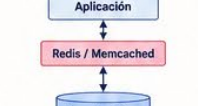
Modelo específico para lecturas optimizadas.



- ✓ Consultas más rápidas.
- ✗ Doble mantenimiento.

15. CACHÉ DE DATOS

Almacena resultados frecuentes en caché.



- ✓ Mejora tiempos de respuesta.
- ✗ Riesgo de datos desactualizados.

PATRONES DE INTEGRACIÓN Y ARQUITECTURA DE DATOS

16. DATABASE PER SERVICE

Cada microservicio tiene su propia base de datos.



- ✓ Aislamiento y autonomía.
- ✗ Consultas entre servicios más difíciles.

17. EVENT SOURCING

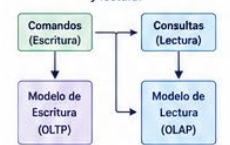
Los cambios se guardan como eventos.



- ✓ Historial completo.
- ✗ Requiere manejo de eventos.

18. CQRS (Command Query Responsibility Segregation)

Separa modelos de escritura y lectura.



- ✓ Escalabilidad y rendimiento.
- ✗ Más complejidad.

19. DATA WAREHOUSE (ESQUEMA ESTRELLA)

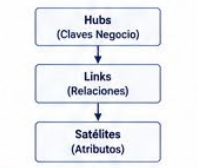
Modelo dimensional para análisis.



- ✓ Consultas analíticas rápidas.
- ✗ No ideal para transacciones.

20. DATA VAULT

Modelo para integrar datos de múltiples fuentes.



- ✓ Histórico completo y flexible.
- ✗ Curva de aprendizaje alta.

¿CUÁNDO USAR CADA PATRÓN?

- ✓ Simplicidad y prototipos → Single Table
- ✓ Aplicaciones OLTP transaccionales → Normalización
- ✓ Lecturas complejas y reportes → Desnormalización / DW
- ✓ Herencia de entidades → CTI / STI / CTT
- ✓ Grandes volúmenes de datos → Particionamiento / Sharding
- ✓ Alta disponibilidad → Replicación
- ✓ Microservicios → Database per Service
- ✓ Trazabilidad y auditoría → Tabla de Auditoría
- ✓ Rendimiento en consultas → Caché / Read Model

RECOMENDACIONES GENERALES

- ✓ Entiende tu dominio y sus reglas.
- ✓ Comienza simple y evoluciona.
- ✓ Aplica normalización hasta 3FN como base.
- ✓ Desnormaliza solo cuando sea necesario (mide antes y después).
- ✓ Documenta tu modelo y decisiones.
- ✓ Revisa índices y planes de ejecución.
- ✓ Monitorea y ajusta continuamente.

FLUJO DE DISEÑO SUGERIDO



EN RESUMEN

Los patrones de diseño de bases de datos son herramientas probadas que nos permiten construir sistemas de datos robustos, escalables y alineados al negocio.



BUEN DISEÑO DE DATOS = MEJOR RENDIMIENTO + MENOS COSTOS + MAYOR ESCALABILIDAD + NEGOCIOS MÁS ÁGILES





BASES DE DATOS POR DOMINIO

Diseña datos alineados al negocio, independientes y escalables



1. ¿QUÉ ES UNA BASE DE DATOS POR DOMINIO?

Es una base de datos que pertenece exclusivamente a un dominio (o bounded context). Cada microservicio es dueño total de sus datos.

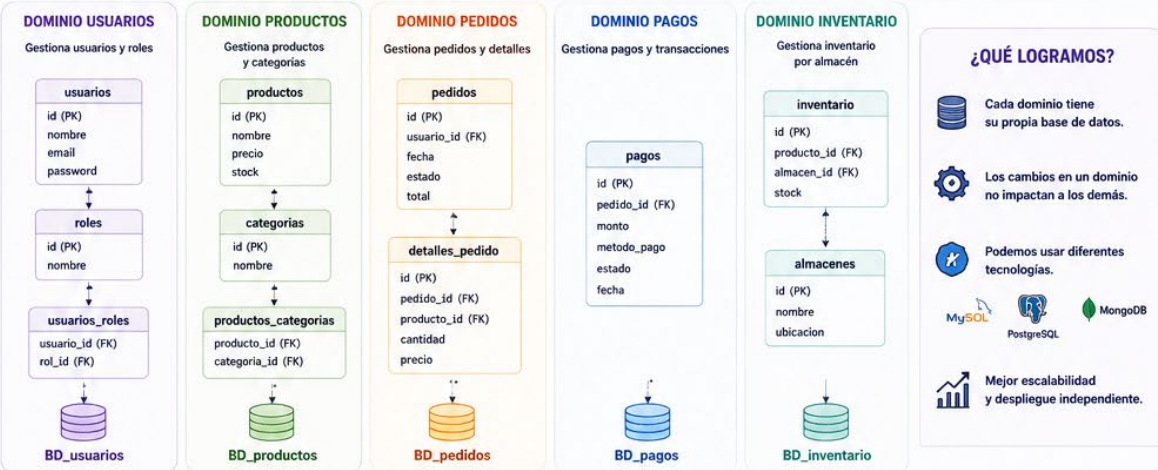
- ✓ Independencia total
- ✓ Evita acoplamiento
- ✓ Permite evolucionar sin afectar a otros servicios
- ✓ Tecnologías diferentes por dominio
- ✓ Mejor seguridad y escalabilidad



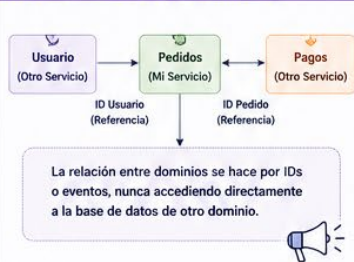
2. PASOS PARA DISEÑAR BASES DE DATOS POR DOMINIO

- 1. Identifica los Dominios**
Analiza el negocio y define los subdominios (o bounded contexts).
- 2. Define Responsabilidades**
Determina qué datos y reglas pertenecen a cada dominio.
- 3. Diseña el Modelo de Datos**
Crea el modelo conceptual y luego el modelo lógico para cada dominio de forma independiente.
- 4. Elige la Tecnología Adecuada**
Selecciona el motor de base de datos que mejor se adapte a las necesidades del dominio.
- 5. Implementa Aislamiento**
Cada dominio debe tener su propia base de datos, esquema o instancia.
- 6. Controla Accesos y Seguridad**
Solo el microservicio dueño puede acceder y modificar su base de datos.

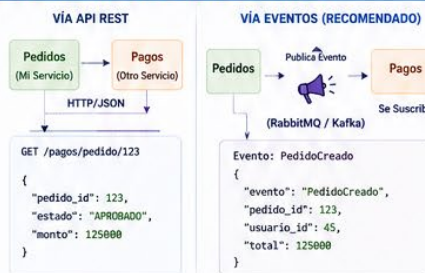
3. EJEMPLO: SISTEMA DE COMERCIO ELECTRÓNICO



4. RELACIÓN ENTRE DOMINIOS



5. COMUNICACIÓN ENTRE DOMINIOS



6. TECNOLOGÍAS POR DOMINIO (EJEMPLOS)



7. BUENAS PRÁCTICAS

- ✓ Diseña el modelo de datos alineado a las reglas del dominio.
- ✓ Cada microservicio es dueño de su base de datos (ownership total).
- ✓ No compartas la base de datos entre servicios.
- ✓ Usa migraciones independientes por dominio.
- ✓ Respaldar y monitorear cada base de datos por separado.
- ✓ Usa eventos o APIs para comunicarte, nunca acceso directo.

8. ANTI-PATRONES A EVITAR

- ✗ Usar una sola base de datos para todos los microservicios.
- ✗ Acceder directamente a la base de datos de otro servicio.
- ✗ Compartir esquemas o tablas entre dominios.
- ✗ Tener relaciones físicas entre bases de datos de diferentes dominios.
- ✗ Ignorar la evolución del modelo de datos del dominio.

9. FLUJO COMPLETO RESUMIDO



RECUERDA

Una buena arquitectura de datos por dominio es la base para microservicios exitosos. Menos acoplamiento, más autonomía, mejor evolución.





ARQUITECTURA HEXAGONAL vs CAPAS Y OTRAS ARQUITECTURAS

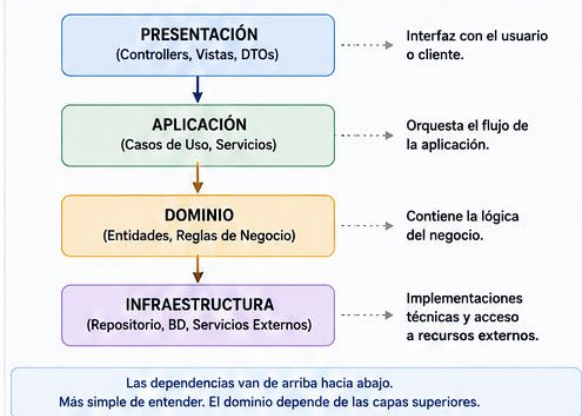


Comparativa visual para elegir la mejor arquitectura según tu contexto

1. ARQUITECTURA HEXAGONAL (PUERTOS Y ADAPTADORES)



2. ARQUITECTURA EN CAPAS (LAYERED)



3. COMPARATIVA RÁPIDA

Aspecto	Hexagonal (Puertos y Adaptadores)	Capas (Layered)	Clean Architecture	Microservicios
Enfoque	Independencia del dominio respecto a tecnologías.	Separación por responsabilidad en niveles.	Independencia absoluta, reglas hacia el centro.	Descomposición por dominio en servicios independientes.
Dependencias	Hacia el centro (el dominio no depende de nada externo).	De arriba hacia abajo.	Hacia el centro (entidades no dependen de nada).	Cada servicio es independiente y se comunica por APIs o mensajería.
Complejidad	Media	Baja	Alta	Alta
Flexibilidad	Alta (fácil cambiar adaptadores).	Media (cambios pueden afectar varias capas).	Muy Alta	Muy Alta
Pruebas	Fácil (dominio aislado).	Media (depende de las capas).	Muy Fácil (muy testeable).	Media/Alta (depende del diseño).
Manejo de tecnología	Tecnología en los bordes (adaptadores).	Tecnología en la capa de infraestructura.	Tecnología en la capa externa (Frameworks & Drivers).	Cada servicio puede usar tecnologías diferentes.
Escalabilidad	Alta (por módulos).	Media	Alta	Muy Alta
Ideal para	Sistemas que deben durar, cambiar de tecnología o ser altamente testeables.	Proyectos pequeños y medianos con requerimientos estables.	Proyectos grandes y críticos que requieren máxima independencia.	Sistemas grandes, equipos distribuidos, escalabilidad independiente.

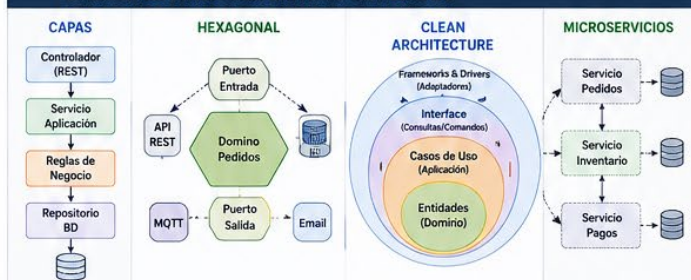
4. OTRAS ARQUITECTURAS POPULARES



5. CUÁNDO USAR CADA UNA



6. EJEMPLO PRÁCTICO: SISTEMA DE PEDIDOS



7. PRINCIPIOS CLAVE EN TODAS

- ✓ Separación de responsabilidades
- ✓ Bajo acoplamiento
- ✓ Alta cohesión
- ✓ Independencia de tecnología
- ✓ Facilidad para pruebas
- ✓ Mantenibilidad y escalabilidad

8. RECOMENDACIÓN FINAL

No existe una única arquitectura perfecta. Elige la que mejor se adapte a tu problema, tu equipo, tu tiempo y tus objetivos.

Puedes comenzar simple y evolucionar cuando el sistema lo requiera.



BUENAS PRÁCTICAS SIEMPRE: DISEÑO POR DOMINIOS + COMUNICACIÓN CLARA + PRUEBAS + CI/CD + MONITOREO





JSON y MQTT

¿QUÉ SON, CÓMO SE USAN Y EN QUÉ ESCENARIOS?



¿QUÉ ES JSON?

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos, legible por humanos y fácil de analizar por máquinas. Es independiente del lenguaje y se usa ampliamente en APIs y aplicaciones web y móviles.

```
{  "nombre": "Juan",  "edad": 28,  "activo": true,  "roles": ["admin", "user"],  "direccion": {    "ciudad": "Bogotá",    "pais": "Colombia"  }}
```

¿CÓMO SE USA?

- Para intercambiar datos entre cliente y servidor.
- En APIs RESTful (peticiones y respuestas).
- Para configuración de aplicaciones.
- Para almacenamiento de datos estructurados.



EJEMPLO DE USO

Respuesta JSON de una API de clima

```
{  "ciudad": "Neiva",  "temperatura": 28.5,  "unidad": "C",  "condicion": "Soleado",  "humedad": 60,  "viento": {    "velocidad": 12,    "direccion": "NE"  }}
```

Este JSON puede ser consumido por cualquier aplicación web, móvil o de escritorio sin importar el lenguaje de programación.

ESCENARIOS DONDE SE USA JSON



APIs Web
Es el formato más común en servicios REST para enviar y recibir datos.



Aplicaciones Móviles
Se usa para intercambiar datos con servidores.



Archivos de Configuración
Muchas aplicaciones usan JSON para guardar configuraciones.



Almacenamiento de Datos
En bases de datos NoSQL (como MongoDB) se usa JSON/BSON.

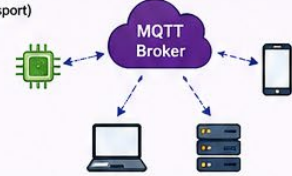


VENTAJAS DE JSON

- ✓ Legible por humanos
- ✓ Ligero y fácil de usar
- ✓ Independiente del lenguaje
- ✓ Muy soportado en la industria

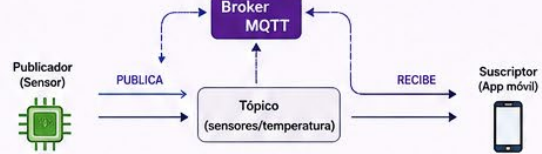
¿QUÉ ES MQTT?

MQTT (Message Queuing Telemetry Transport) es un protocolo ligero de mensajería publicar/suscribir (publish/subscribe), diseñado para dispositivos con pocos recursos y redes con alto retardo o bajo ancho de banda. Funciona sobre TCP/IP.



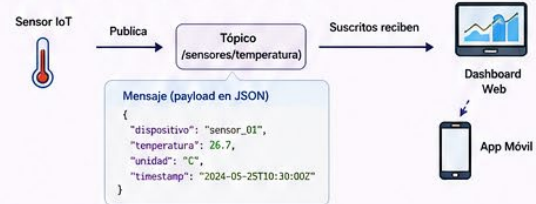
¿CÓMO SE USA?

- Los clientes se conectan a un Broker MQTT.
- Los clientes publican mensajes en TÓPICOS (topics).
- Los clientes se suscriben a tópicos para recibir mensajes.
- El broker distribuye los mensajes a los suscriptores.



EJEMPLO DE USO

Un sensor de temperatura publica datos cada 5 segundos.



ESCENARIOS DONDE SE USA MQTT



Internet de las Cosas (IoT)
Conecta miles de dispositivos que envían datos constantemente.



Monitoreo en tiempo real
Datos de sensores, telemetry, estado de máquinas, vehículos, etc.



Automatización Industrial
Comunicación entre PLCs, sensores y sistemas SCADA.



Aplicaciones Móviles y Chat
Mensajería en tiempo real, notificaciones, presencia.

VENTAJAS DE MQTT

- ✓ Ligero (pocos bytes de overhead)
- ✓ Funciona en redes inestables o de bajo ancho de banda
- ✓ Bajo consumo de energía (ideal para dispositivos IoT)
- ✓ Escalable (miles de clientes conectados)
- ✓ Mensajería en tiempo real

JSON vs MQTT

Característica	JSON	MQTT
Tipo	Formato de datos	Protocolo de mensajería
Propósito	Intercambiar datos	Transportar mensajes
Modelo	Solicitud / Respuesta	Publicar / Suscribir
Transporte	HTTP / HTTPS	TCP/IP
Uso principal	APIs, configuración, datos	IoT, tiempo real, telemetría
Tamaño	Puede ser más grande	Muy ligero
Persistencia	No definida por JSON	Soportada por el broker (QoS)

¿CÓMO SE COMBINAN?

En muchos sistemas IoT, los mensajes MQTT usan JSON como formato del payload.



```
Payload del mensaje (JSON)
{
  "dispositivo": "sensor_01",
  "valor": 23.4,
  "unidad": "C",
  "timestamp": "2024-05-25T10:30:02"
}
```

BUENAS PRÁCTICAS

- Usa tópicos jerárquicos
Ej: hogar/sala/temperatura
- Usa QoS adecuado
0: Máximo 1 vez
1: Al menos 1 vez
2: Exactamente 1 vez
- Mantén los mensajes pequeños
Envía solo lo necesario.
- Usa JSON compacto
Evita espacios innecesarios.
- Seguro siempre
Usa TLS/SSL en MQTT.

EN RESUMEN

JSON es un formato de datos que usamos para estructurar información.

MQTT es un protocolo que usamos para transmitir esa información de forma eficiente y en tiempo real. Juntos son una combinación poderosa para aplicaciones modernas e IoT.





DOMINIOS, MICROSERVICIOS Y PATRONES DE DISEÑO

¿QUÉ HACEN Y CÓMO SE INTEGRAN?



El dominio define **QUÉ** hace el sistema (negocio).
Los patrones definen **CÓMO** implementamos la solución de forma mantenible.

1. ¿QUÉ ES UN DOMINIO?

Un dominio es un área del negocio que agrupa procesos, reglas y datos relacionados con una misma función.

Ejemplo: Tienda Virtual (E-commerce)



Cada subdominio puede convertirse en un microservicio independiente.

2. RELACIÓN ENTRE DOMINIOS, MICROSERVICIOS Y PATRONES



3. EJEMPLOS DE DOMINIOS Y PATRONES UTILIZADOS



4. INTEGRACIÓN COMPLETA



5. ARQUITECTURA DENTRO DE UN MICROSERVICIO



6. EJEMPLO EN SISTEMA ACADÉMICO (SENA)

Dominio	Microservicio	Patrones recomendados
Alumnos	Gestión de alumnos	Factory, Repository
Instructores	Gestión de instructores	Singleton, Repository
Matrículas	Gestión de matrículas	Strategy, Builder
Evaluaciones	Gestión de notas	Observer, Strategy
Notificaciones	Envío de correos y SMS	Observer, Facade
Autenticación	Login y seguridad	Singleton, Factory

7. IDEA CLAVE - SECUENCIA LÓGICA



★ Dominio + Microservicios + Arquitectura + Patrones = Aplicaciones escalables, mantenibles y alineadas al negocio. 🚀



¿QUÉ ES UN DOMINIO EN MICROSERVICIOS?

Un dominio es un **área del negocio** que agrupa procesos, reglas y datos relacionados con una misma función. Es la base para dividir una aplicación en microservicios independientes.

1. EJEMPLO DE NEGOCIO



Imagina un sistema de comercio electrónico. El dominio principal es:

COMERCIO ELECTRÓNICO (E-commerce)

Este dominio se divide en subdominios que pueden convertirse en microservicios.

SUBDOMINIOS (POSIBLES MICROSERVICIOS)



Usuarios



Productos



Carrito



Pagos



Envíos



Facturación



Reseñas



Reportes

2. ¿QUÉ HACE UN DOMINIO?

El dominio define todo lo relacionado con esa parte del negocio.



Reglas del negocio
Define las normas y políticas.



Datos que administra
Gestiona su propia información.



Procesos que ejecuta
Realiza tareas específicas.



Eventos que genera
Notifica cambios o acciones.

Ejemplo: Dominio PEDIDOS



- ✓ Crear un pedido
- ✓ Cancelar un pedido
- ✓ Confirmar un pedido
- ✓ Cambiar su estado

Estas reglas solo pertenecen al microservicio PEDIDOS.

3. ¿POR QUÉ ES IMPORTANTE?

Una de las reglas más importantes de la arquitectura de microservicios es:

“Cada microservicio debe encargarse de un único dominio o subdominio del negocio.”



Evita servicios demasiado grandes y complejos.



Permite escalar de forma independiente cada servicio.



Facilita el mantenimiento, despliegue y evolución del sistema.

4. EJEMPLOS DE DOMINIOS EN DIFERENTES NEGOCIOS



BANCO

- Clientes
- Cuentas
- Transferencias
- Créditos
- Tarjetas
- Seguridad
- Notificaciones

Cada uno puede ser un microservicio independiente



UNIVERSIDAD

- Estudiantes
- Docentes
- Matrículas
- Cursos
- Horarios
- Calificaciones
- Pagos

Cada uno puede ser un microservicio independiente



HOSPITAL

- Pacientes
- Citas
- Historias clínicas
- Facturación
- Laboratorios
- Farmacia
- Notificaciones

Cada uno puede ser un microservicio independiente



5. DOMINIO vs. MICROSERVICIO

DOMINIO	MICROSERVICIO
Representa una parte del negocio	Implementa ese dominio mediante software
Contiene reglas y procesos	Contiene código, API y base de datos
Es un concepto de negocio	Es un componente técnico
Ejemplo: Pagos	Ejemplo: Servicio de Pagos

6. DOMAIN-DRIVEN DESIGN (DDD)

DDD propone diseñar el software alrededor del negocio. Divide el dominio en Bounded Contexts (contextos delimitados), y cada contexto suele convertirse en un microservicio.



Cada microservicio tiene:

- ✓ Su propia base de datos.
- ✓ Sus propias reglas de negocio.
- ✓ Su propia API.
- ✓ Independencia para desplegarse y escalar.



EN RESUMEN

Un dominio en microservicios es una parte del negocio con una responsabilidad bien definida. Al identificar correctamente los dominios y subdominios, es posible construir microservicios independientes, mantenibles y escalables, donde cada servicio se enfoca en resolver un único problema del negocio.

